МІНІСТЕРСТВО ОСВІТИ ТА НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
«КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ ІМЕНІ ІГОРЯ СІКОРСЬКОГО»
ФАКУЛЬТЕТ ІНФОРМАТИКИ ТА ОБЧИСЛЮВАЛЬНОЇ ТЕХНІКИ
КАФЕДРА ОБЧИСЛЮВАЛЬНОЇ ТЕХНІКИ

Лабораторна робота №2

з дисципліни «Об’єктно орієнтовне програмування»

на тему «Розробка графічного редактора об’єктів»

Виконав:
студент групи ІМ-о51
Романов Єгор Дмитрович

Перевірив:
Рекечинський Д.О.

Київ 2026

Завдання

1. Створити у середовищі розробки новий проект з ім’ям Lab2. 2.

Скомпілювати проект і отримати виконуваний файл програми. 3. Перевірити роботу програми. За потреби налагодити програму. 4. Проаналізувати та прокоментувати результати та вихідний текст програми. 5. Оформити звіт

Завдання згідно варіанту

Масив для динамічних об’єктів типу Shape - Динамічний

Гумовий слід - пунктирна лінія чорного кольору

Прямокутник - увід по двум протилежник кутам, чорний контур з жовтим заповненням

Еліпс - увід від центру до одного з кутів , чорний контур без заповнення

Позначка поточного типу об’єкта - в заголовку вікна

Код

==========

File: main.py
Path: E:\OOP-Labs\lab_2\main.py

==========

```
import tkinter as tk
from .modules import Canvas, Options


class App(tk.Tk):
    def __init__(self, screenName=None, baseName=None, className="Tk", useTk=True, sync=False, use=None):
        super().__init__(screenName, baseName, className, useTk, sync, use)

        self._canvas = Canvas(self, render=False)
        self._options = Options(self, self._canvas._switch_shape_cb)
        self._canvas.render()

if __name__ == "__main__":
    App(screenName="canvas", baseName="lab-2").mainloop()
```

==========

File: tkinter_showcase.py
Path: E:\OOP-Labs\lab_2\tkinter_showcase.py

==========

```
import tkinter as tk


root = tk.Tk()
root.geometry("300x500")

canvas_editor = tk.Canvas(root, height=600, width=900)

def paint( event ):

    # Co-ordinates.
    x1, y1, x2, y2 = ( event.x - 3 ),( event.y - 3 ), ( event.x + 3 ),( event.y + 3 )

    # Colour
    Colour = "#000fff000"

    # specify type of display
    canvas_editor.create_line( x1, y1, x2,
                y2, fill = Colour )


canvas_editor.bind( "<B1-Motion>", paint )
canvas_editor.pack(expand=True)

root.mainloop()
```

==========

File: __init__.py
Path: E:\OOP-Labs\lab_2__init__.py

==========

```
from .modules.shapes_implementations.shape_abstract import
ShapeAbstract
from .modules.shapes_implementations.shape_point import ShapePoint
from .modules.shapes_implementations.shape_line import ShapeLine
from .modules.shapes_implementations.shape_rect import ShapeRect
from .modules.shapes_implementations.shape_ellipse import ShapeEllipse

from .modules.shapes_implementations.editor_canvas import
EditorCanvas
```

```
============================================================
File: canvas.py
Path: E:\OOP-Labs\lab_2\modules\canvas.py
============================================================
import tkinter as tk
from .shapes_implementations import ShapeFactory, ShapeAbstract,
ShapesKeys
from typing import List

class Canvas:
    def __init__(self,
        root: tk.Misc,
        render: bool = False
    ):
        self._canvas = tk.Canvas(root, bg="white")

        self._canvas.bind("<ButtonPress>", self._mouse_start_drawing_cb)
        self._canvas.bind("<Motion>", self._mouse_drawing_cb)
        self._canvas.bind("<ButtonRelease>", self._mouse_stop_drawing_cb)

        self._is_drawing = False

        self._curr_shape_key: ShapesKeys | None = None
        self._curr_shape: ShapeAbstract | None = None
        self._shapes_stack: List[ShapeAbstract] = []

        # rubber path
        self._rubber_path_canvas_internal_id: int | None = None

        # coords
        self._x_start: int | None = None
        self._y_start: int | None = None

        if render:
            self.render()

    @property
    def curr_shape(self) -> ShapesKeys | None:
        return self._curr_shape_key

    def render(self) -> None:
        self._canvas.pack()
```

```
    def _create_rubber_path(self, x_start: int, y_start: int) -> None:
        self._x_start = x_start
        self._y_start = y_start
        self._rubber_path_canvas_internal_id = self._canvas.create_rectangle(
            self._x_start, self._y_start,
            self._x_start, self._y_start,
            outline="black"
        )

    def _update_rubber_path(self, end_x: int, end_y: int) -> None:
        self._canvas.coords(
            self._rubber_path_canvas_internal_id,
            self._x_start, self._y_start,
            end_x, end_y
        )

    def _remove_rubber_path(self) -> None:
        """handles clear of `_rubber_path_canvas_internal_id`"""

        if self._rubber_path_canvas_internal_id is None:
            return

        self._canvas.delete(self._rubber_path_canvas_internal_id)
        self._rubber_path_canvas_internal_id = None

    def _switch_shape_cb(self, key: ShapesKeys | None) -> None:
        if key is None:
            self._curr_shape = None
            return

        self._curr_shape_key = key
        self._reinstantiate_shape()

    def _reinstantiate_shape(self) -> None:
        if self._curr_shape_key is not None:
            self._curr_shape =
ShapeFactory().instantiate_shape(self._curr_shape_key, self._canvas)
            return

        self._curr_shape = None

    def _mouse_start_drawing_cb(self, event: tk.Event) -> None:
        self._x_start = event.x
        self._y_start = event.y
```

```
        self._is_drawing = True
        self._create_rubber_path(self._x_start, self._y_start)

    def _mouse_drawing_cb(self, event: tk.Event) -> None:
        if self._is_drawing:
            if self._curr_shape:
                self._curr_shape.update_position(
                    self._x_start, event.x,
                    self._y_start, event.y
                )
            self._update_rubber_path(event.x, event.y)

    def _mouse_stop_drawing_cb(self, event: tk.Event) -> None:
        self._x_start = None
        self._y_start = None

        self._is_drawing = False

        self._remove_rubber_path()

        if self._curr_shape:
            self._curr_shape.render()
            self._shapes_stack.append(self._curr_shape)

            self._reinstantiate_shape()

    def undo(self) -> None:
        if len(self._shapes_stack) > 0:
            shape_to_undo = self._shapes_stack.pop()
            shape_to_undo.dispose()

            self._reinstantiate_shape()
```

==========

File: options.py
Path: E:\OOP-Labs\lab_2\modules\options.py

==========

```
import tkinter as tk

from .options_implementations import FileOption, RefOption,
ShapeOption
from .shapes_implementations import SwitchShapeCb
```

```
class Options:
    def __init__(self, root: tk.Misc, switch_shape_cb: SwitchShapeCb):
        self.root = root
        self.toolbar = tk.Frame(root, bd=1, relief=tk.SUNKEN)
        self.toolbar.pack(side=tk.TOP, fill=tk.X)

        self.file_option = FileOption(root)
        self.ref_option = RefOption(root)
        self.shape_option = ShapeOption(root, switch_shape_cb)

        self.file_option.render(self.toolbar)
        self.ref_option.render(self.toolbar)
        self.shape_option.render(self.toolbar)
```

```
============================================================
File: __init__.py
Path: E:\OOP-Labs\lab_2\modules\__init__.py
============================================================
```

```
from .shapes_implementations import *
from .options_implementations import *

from .canvas import Canvas
from .options import Options

__all__ = [
    "Canvas",
    "Options",
    "ShapeFactory",
    "ShapeAbstract",
    "ShapePoint",
    "ShapeLine",
    "ShapeRect",
    "ShapeEllipse",
    "EditorCanvas",
    "ShapesKeys",
    "SHAPES_MAP",
]
```

```
============================================================
File: file_option.py
Path: E:\OOP-Labs\lab_2\modules\options_implementations\file_option.py
============================================================
```

```
import tkinter as tk

from .option import BaseDropdownOptions


class FileOption(BaseDropdownOptions):
    def __init__(self, root: tk.Misc):
        super().__init__(root, {"Close": lambda _: root.destroy()}, ["Close"])
        self._label = "File"
        self.button = tk.Button(root, text=self._label, command=lambda:
self._menu.post(root.winfo_rootx() + 10, root.winfo_rooty() + 30))

    def render(self, parent: tk.Misc | None = None) -> None:
        target = parent if parent is not None else self.root
        self.button.pack(in_=target, side=tk.LEFT, padx=(0, 8), pady=4)

    def open_file(self, *args, **kwargs):
        return None
```

```
============================================================
File: option.py
Path: E:\OOP-Labs\lab_2\modules\options_implementations\option.py
============================================================
import tkinter as tk
from abc import ABC, abstractmethod
from typing import Any, Callable, Dict, List


class BaseDropdownOptions(ABC):
    def __init__(self, root: tk.Misc, options_cbs: Dict[str, Callable[[Any],
None]], readable_options: List[str] | None = None):
        self.root = root
        self._label = "Menu"
        self._options = list(options_cbs.keys()) if readable_options is None else
readable_options
        self._options_cbs = options_cbs
        self._menu = tk.Menu(root, tearoff=0)
        self._attach_menu_items()

    def _attach_menu_items(self) -> None:
        for option in self._options:
            callback = self._options_cbs.get(option)
            self._menu.add_command(label=option, command=lambda
```

```
value=option: self._select_option(value))

    def _select_option(self, option: str) -> None:
        callback = self._options_cbs.get(option)
        if callback is not None:
            callback(option)

    @property
    def chosen_option(self) -> str:
        return self._label

    @abstractmethod
    def render(self, parent: tk.Misc | None = None) -> None:
        ...
```

==========

File: ref_option.py
Path: E:\OOP-Labs\lab_2\modules\options_implementations\ref_option.py

==========

```
import tkinter as tk

from .option import BaseDropdownOptions


class RefOption(BaseDropdownOptions):
    def __init__(self, root: tk.Misc):
        super().__init__(root, {"Work is done by Yehor Romanov IM-o51":
lambda _: None}, ["Work is done by Yehor Romanov IM-o51"])
        self._label = "Reference"
        self.button = tk.Button(root, text=self._label, command=lambda:
self._menu.post(root.winfo_rootx() + 10, root.winfo_rooty() + 30))

    def render(self, parent: tk.Misc | None = None) -> None:
        target = parent if parent is not None else self.root
        self.button.pack(in_=target, side=tk.LEFT, padx=(0, 8), pady=4)

    def refresh(self, *args, **kwargs):
        return None
```

==========

File: shape_option.py
Path:

E:\OOP-Labs\lab_2\modules\options_implementations\shape_option.py

```
import tkinter as tk

from .option import BaseDropdownOptions
from ..shapes_implementations import ShapesKeys

class ShapeOption(BaseDropdownOptions):
    def __init__(self, root: tk.Misc, switch_shape_cb):
        self._switch_shape_cb = switch_shape_cb
        options = {
            "Rectangle": lambda _, shape_name="RECTANGLE":
self._set_shape(shape_name),
            "Point": lambda _, shape_name="POINT":
self._set_shape(shape_name),
            "Line": lambda _, shape_name="LINE":
self._set_shape(shape_name),
            "Ellipse": lambda _, shape_name="ELLIPSE":
self._set_shape(shape_name),
            "No Shape": lambda _: self._set_shape(None),
        }
        super().__init__(root, options, ["Rectangle", "Point", "Line",
"Ellipse", "No Shape"])
        self._label = "No Shape"
        self.button = tk.Button(root, text=self._label, command=lambda:
self._menu.post(root.winfo_rootx() + 10, root.winfo_rooty() + 30))

    def render(self, parent: tk.Misc | None = None) -> None:
        target = parent if parent is not None else self.root
        self.button.pack(in_=target, side=tk.LEFT, padx=(0, 8), pady=4)

    def _set_shape(self, shape_name: ShapesKeys | None):
        if shape_name is None:
            self._label = "No Shape"
        else:
            self._label = shape_name.title()

        self.button.config(text=self._label)
        self._switch_shape_cb(shape_name)
```

File: __init__.py
Path: E:\OOP-Labs\lab_2\modules\options_implementations__init__.py

```
============================================================
from .file_option import FileOption
from .ref_option import RefOption
from .shape_option import ShapeOption
from .option import BaseDropdownOptions


============================================================
File: shape_abstract.py
Path:
E:\OOP-Labs\lab_2\modules\shapes_implementations\shape_abstract.py
============================================================
from abc import ABC, abstractmethod
from typing import Tuple
import tkinter as tk


class ShapeAbstract(ABC):
    def __init__(self, canvas: tk.Canvas):
        self._canvas = canvas
        self._x1 = self._x2 = self._y1 = self._y2 = None

        self._internal_canvas_id: int | None = None
        self._is_rendered = False

    @property
    def internal_canvas_id(self) -> int | None:
        return self._internal_canvas_id

    @property
    def is_rendered(self) -> bool:
        return self._is_rendered

    @is_rendered.setter
    def is_rendered(self, value: bool) -> None:
        if not isinstance(value, bool):
            raise ValueError(f"`is_rendered` attribute is boolean only, received
{type(value)}")

        if value is True:
            self.render()

        if value is False and self._internal_canvas_id is not None:
            self._canvas.delete(self._internal_canvas_id)
```

```
            self._internal_canvas_id = None
            self._is_rendered = False
            return

        raise ValueError("Couldn't dispose not rendered shape!")

    @abstractmethod
    def render(self) -> None:
        """Must handle internal `_is_rendered` and `_internal_canvas_id`
attributes change"""
        ...

    def dispose(self) -> None:
        self.is_rendered = False # activates setter disposing flow

    def update_position(self, *coords: int) -> None:
        """*cords: List[x1, x2, y1, y2]"""

        if len(coords) != 4:
            raise ValueError(f"4 positioning parameters are expected, received
{len(coords)}")

        self._x1, self._x2, self._y1, self._y2 = coords
```

==========

File: shape_ellipse.py
Path:
E:\OOP-Labs\lab_2\modules\shapes_implementations\shape_ellipse.py

==========

```
import tkinter as tk
from .shape_abstract import ShapeAbstract


class ShapeEllipse(ShapeAbstract):
    _DEFAULT_COLOR = "black"

    def __init__(self, canvas):
        super().__init__(canvas)

    def render(self) -> None:
        cx, cy = self._x1, self._y1
        rx = abs(self._x2 - cx)
        ry = abs(self._y2 - cy)
```

```
        self._internal_canvas_id = self._canvas.create_oval(
            cx - rx, cy - ry,
            cx + rx, cy + ry,
            outline=self._DEFAULT_COLOR,
        )


============================================================
File: shape_factory.py
Path:
E:\OOP-Labs\lab_2\modules\shapes_implementations\shape_factory.py
============================================================
from typing import Dict, Literal, Type

from .shape_abstract import ShapeAbstract
from .shape_ellipse import ShapeEllipse
from .shape_line import ShapeLine
from .shape_point import ShapePoint
from .shape_rect import ShapeRect
from .types import ShapesKeys

SHAPES_MAP: Dict[ShapesKeys, Type[ShapeAbstract]] = {
    "RECTANGLE": ShapeRect,
    "POINT": ShapePoint,
    "LINE": ShapeLine,
    "ELLIPSE": ShapeEllipse,
}


class ShapeFactory:
    @staticmethod
    def instantiate_shape(key: ShapesKeys, *args, **kwargs) ->
ShapeAbstract:
        ShapeClass = SHAPES_MAP.get(key)

        if not ShapeClass:
            raise ValueError("Couldn't find desired shape")
        if not issubclass(ShapeClass, ShapeAbstract):
            raise ValueError("This shape isn't supported. Shape must be a heir
of `ShapeAbstract`")

        return ShapeClass(*args, **kwargs)
```

```
============================================================
File: shape_line.py
Path: E:\OOP-Labs\lab_2\modules\shapes_implementations\shape_line.py
============================================================
import tkinter as tk
from .shape_abstract import ShapeAbstract


class ShapeLine(ShapeAbstract):
    _DEFAULT_COLOR = "black"

    def __init__(self, canvas):
        super().__init__(canvas)

    def render(self) -> None:
        self._internal_canvas_id = self._canvas.create_line(
            self._x1, self._y1,
            self._x2, self._y2,
            fill=self._DEFAULT_COLOR,
        )
        self._is_rendered = True


============================================================
File: shape_point.py
Path:
E:\OOP-Labs\lab_2\modules\shapes_implementations\shape_point.py
============================================================
import tkinter as tk
from .shape_abstract import ShapeAbstract


class ShapePoint(ShapeAbstract):
    _DEFAULT_COLOR = "blue"

    def __init__(self, canvas):
        super().__init__(canvas)

    def render(self) -> None:
        self._internal_canvas_id = self._canvas.create_oval(
            self._x1 - 2, self._y1 - 2,
            self._x1 + 2, self._y1 + 2,
            fill=self._DEFAULT_COLOR,
```

```
            outline=self._DEFAULT_COLOR,
        )
        self._is_rendered = True


================================================================
File: shape_rect.py
Path: E:\OOP-Labs\lab_2\modules\shapes_implementations\shape_rect.py
================================================================
import tkinter as tk
from .shape_abstract import ShapeAbstract


class ShapeRect(ShapeAbstract):
    _DEFAULT_COLOR = "yellow"

    def __init__(self, canvas):
        super().__init__(canvas)

    def render(self) -> None:
        self._internal_canvas_id = self._canvas.create_rectangle(
            self._x1, self._y1,
            self._x2, self._y2,
            fill=self._DEFAULT_COLOR
        )
        self._is_rendered = True

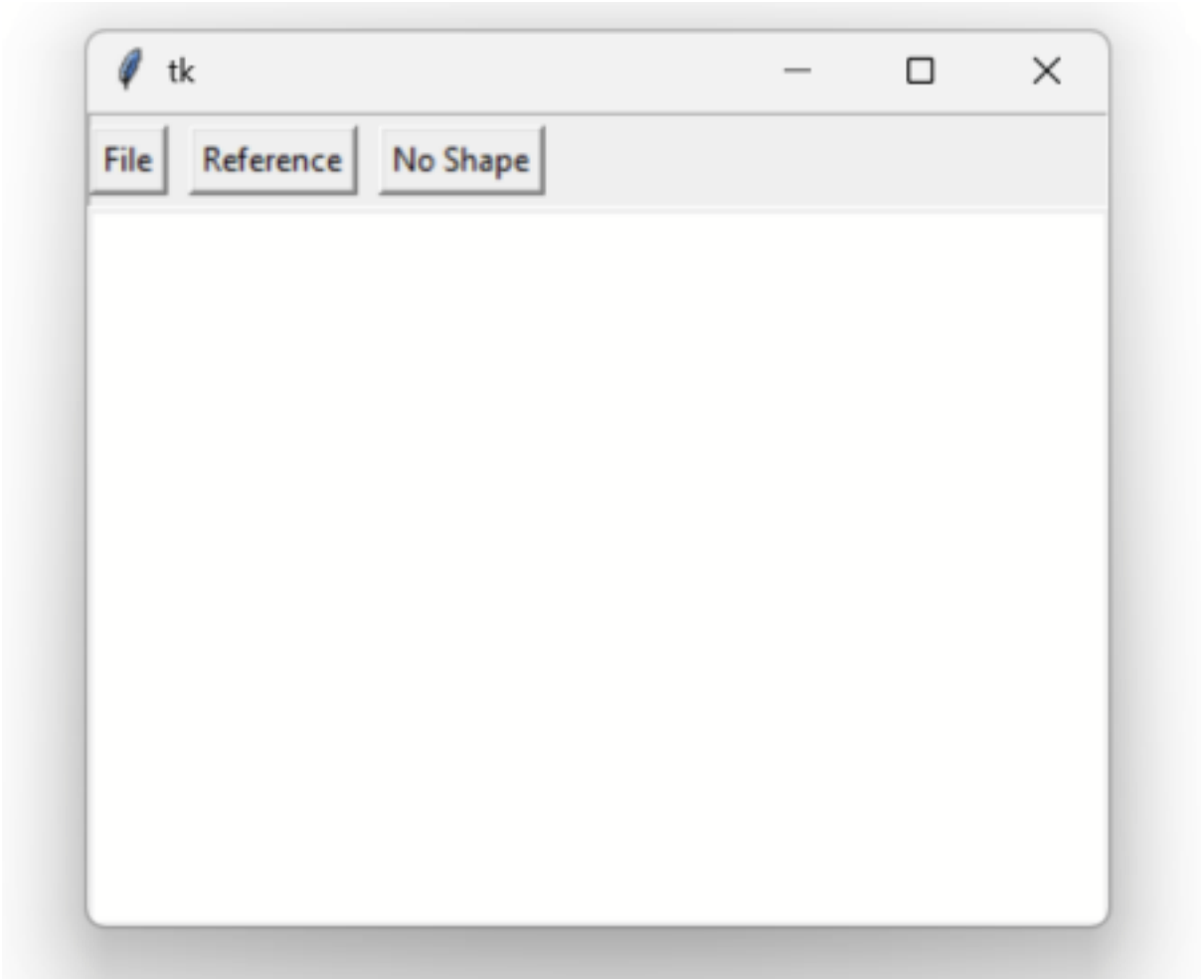

================================================================
File: types.py
Path: E:\OOP-Labs\lab_2\modules\shapes_implementations\types.py
================================================================
from typing import Literal, Callable

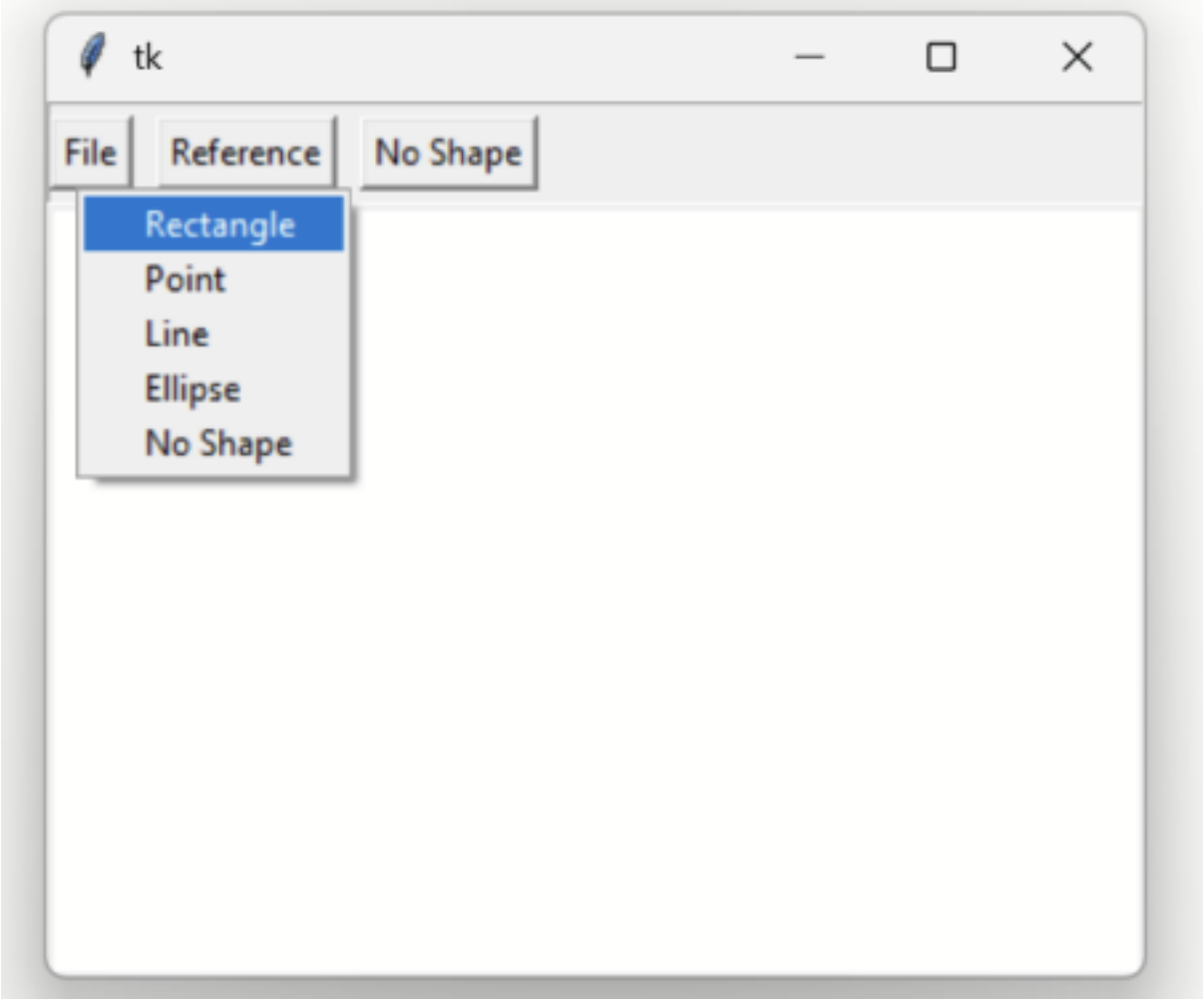
ShapesKeys = Literal["RECTANGLE", "POINT", "LINE", "ELLIPSE"]
SwitchShapeCb = Callable[[ShapesKeys], None]

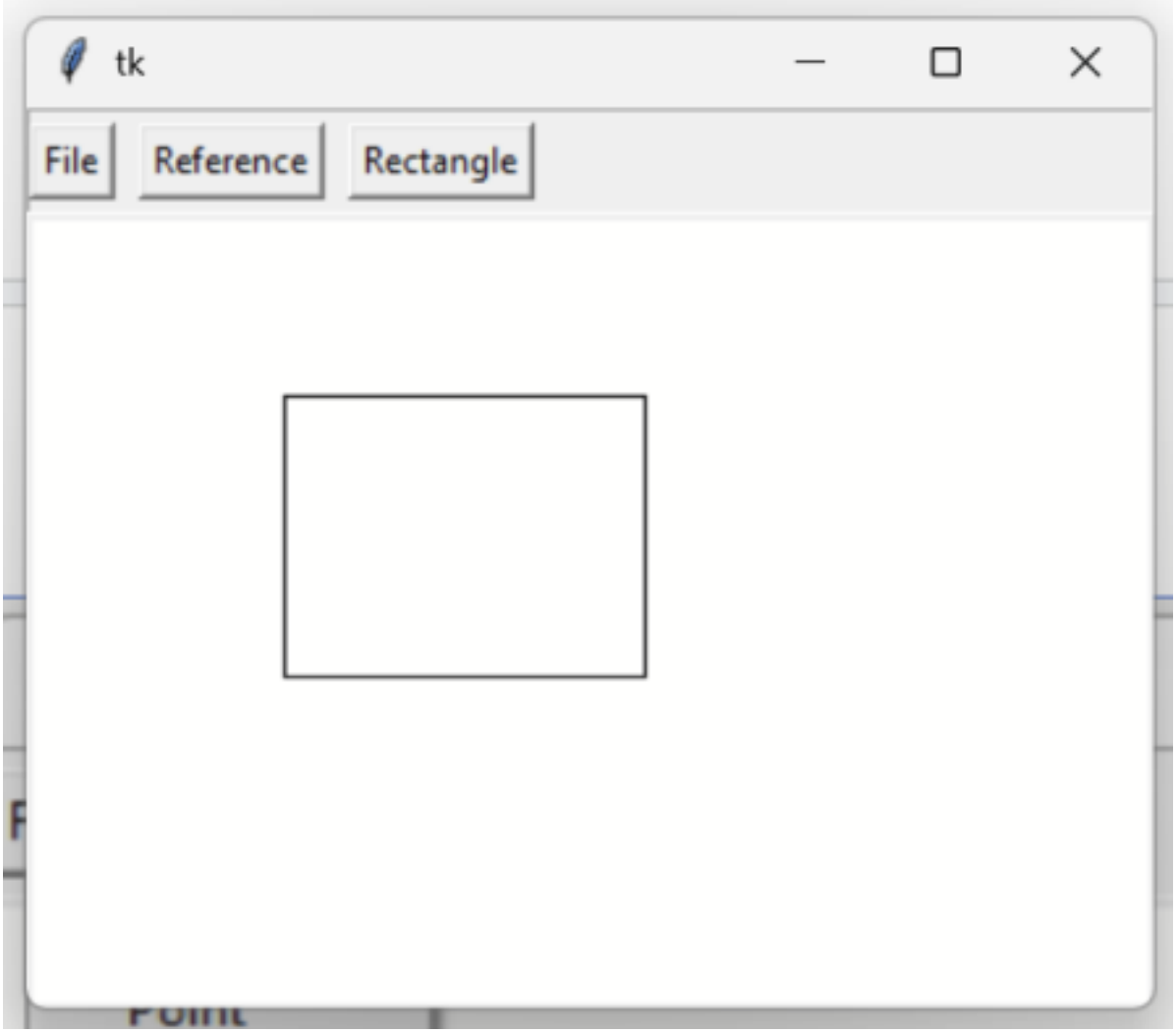

================================================================
File: __init__.py
Path: E:\OOP-Labs\lab_2\modules\shapes_implementations\__init__.py
================================================================
from .shape_abstract import ShapeAbstract
from .shape_ellipse import ShapeEllipse
```

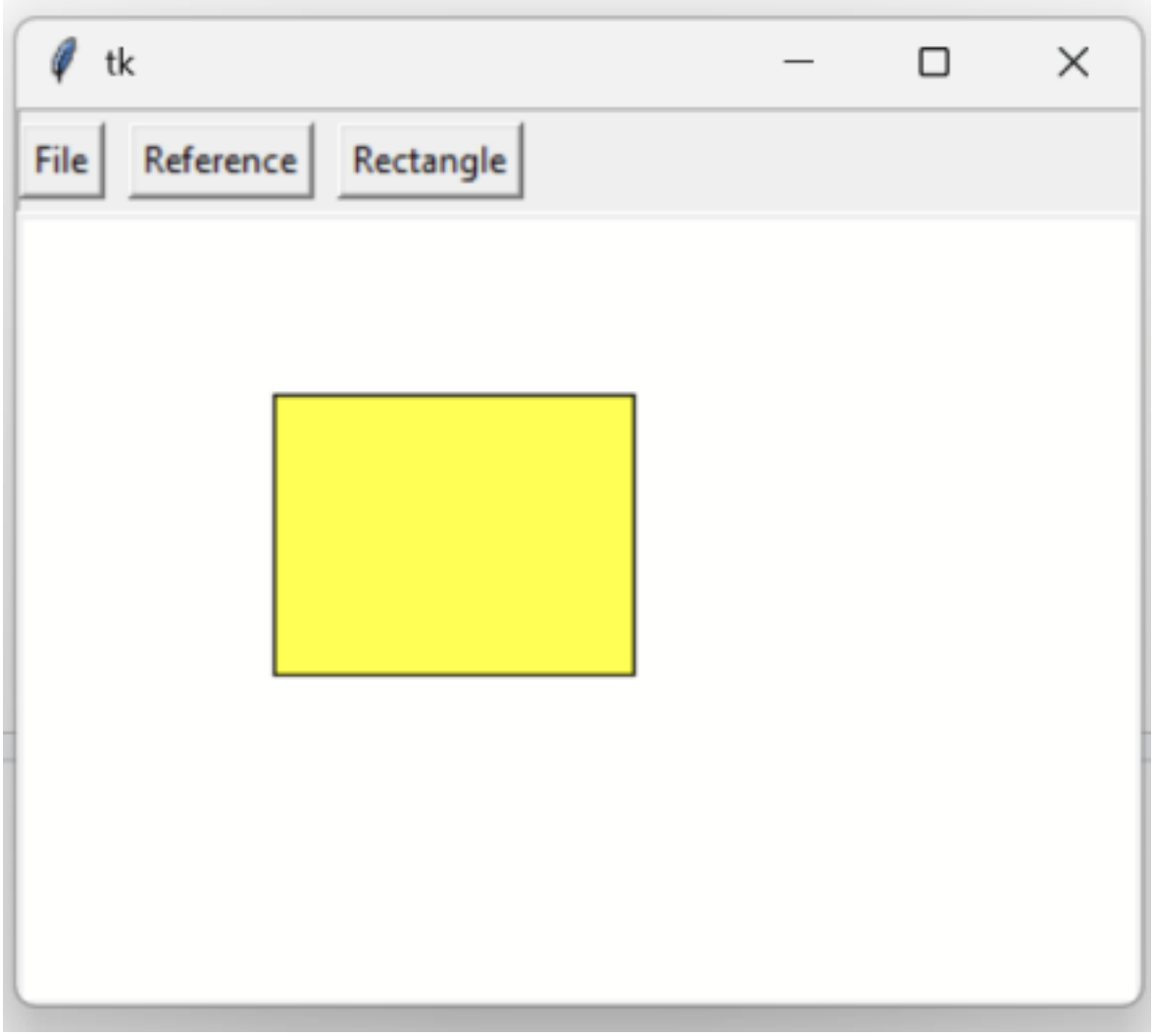
```
from .shape_line import ShapeLine
from .shape_point import ShapePoint
from .shape_rect import ShapeRect
from .shape_factory import ShapeFactory, SHAPES_MAP
from .types import ShapesKeys, SwitchShapeCb
```

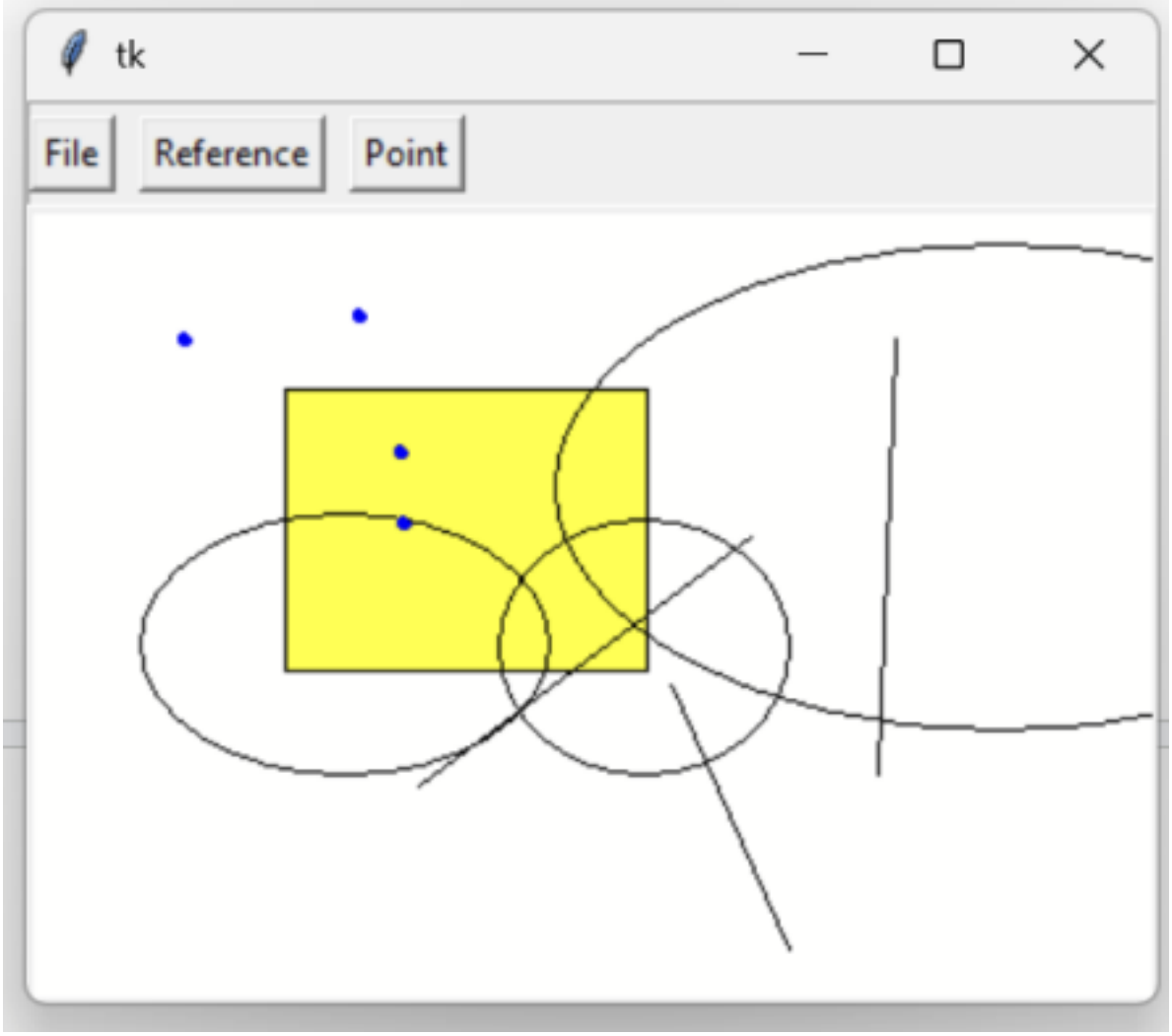
Тестування

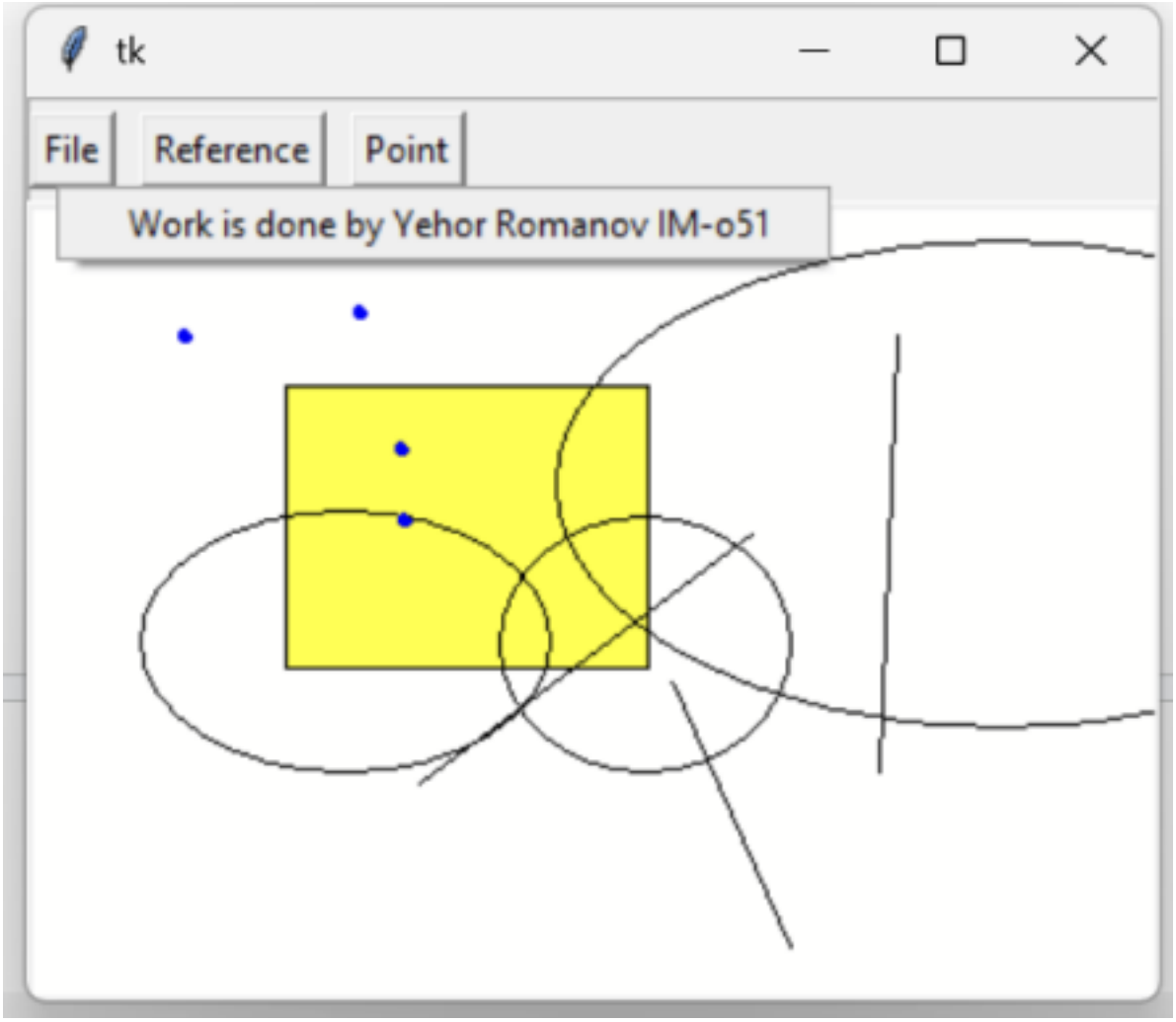


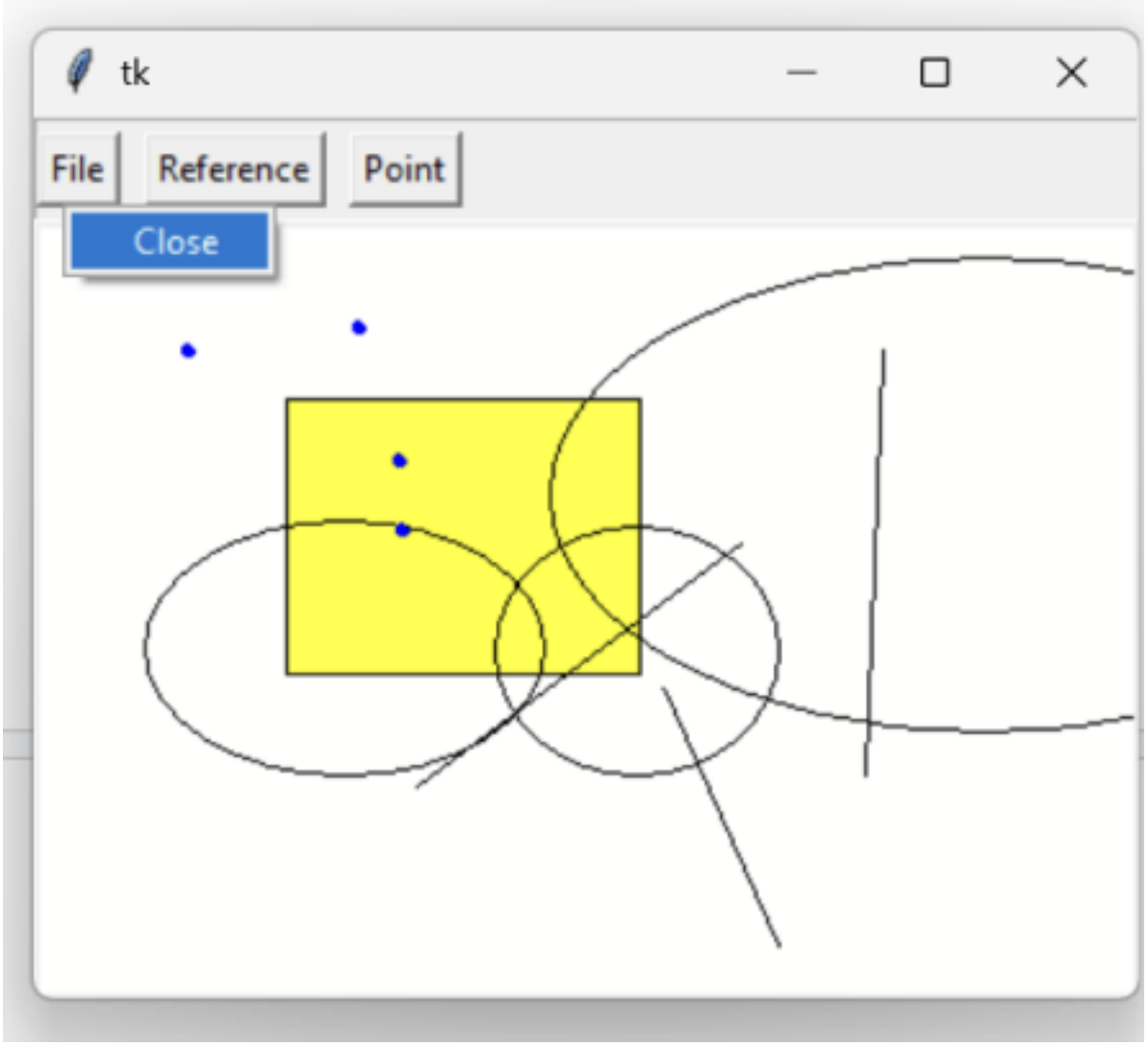

tk
File
Reference
No Shape
Rectangle
Point
Line
Ellipse
No Shape


tk
File
Reference
Rectangle


tk
File
Reference
Rectangle


tk
File
Reference
Point


tk
File
Reference
Point
Work is done by Yehor Romanov IM-o51



Висновки

Під час виконання лабораторної роботи №2 було розроблено графічний редактор об’єктів, у якому реалізовано динамічний масив для зберігання об’єктів типу Shape, відображення гумового сліду у вигляді чорної пунктирної лінії, малювання прямокутників і еліпсів із відповідним заповненням та контурами, а також відображення позначки поточного типу об’єкта в заголовку вікна.